TP 2.2 - GENERADORES DE NÚMEROS PSEUDOALEATORIOS DE DISTINTAS DISTRIBUCIONES

Brizio Augusto
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
brizioaugusto@gmail.com

Casermeiro Gonzalo Nicolás
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
gonzacasermeiro@gmail.com

Messina Tiziano Leonel
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
tizilmessina@gmail.com

De la Rosa Valentin
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
mferreyra079@gmail.com

June 4, 2026

ABSTRACT

Este trabajo continúa el estudio de generadores pseudoaleatorios iniciado en el TP 2.1. A partir de números uniformes en el intervalo $[0, 1)$ se implementaron generadores para nueve distribuciones de probabilidad: uniforme, exponencial, gamma, normal, Pascal, binomial, hipergeométrica, Poisson y empírica discreta. El objetivo fue transformar una fuente uniforme en variables aleatorias con comportamientos probabilísticos distintos, utilizando transformación inversa, composición de variables, ensayos de Bernoulli, muestreo sin reemplazo, el método multiplicativo y el método de Box-Muller.

Para validar las simulaciones se generaron $n = 10\,000$ valores por distribución y se compararon la media y la varianza observadas contra sus valores teóricos. Además, para distribuciones continuas se aplicó una prueba de Kolmogorov-Smirnov y para distribuciones discretas una prueba chi-cuadrado de bondad de ajuste. Los resultados muestran errores relativos bajos en media y varianza, y todos los estadísticos de ajuste quedaron dentro de los criterios de aceptación definidos para el experimento. Esto confirma que las transformaciones implementadas producen secuencias compatibles con las distribuciones esperadas.

1 Introducción

En el TP 2.1 se estudió la generación de números pseudoaleatorios uniformes y la necesidad de verificar su calidad estadística. Sin embargo, muchas simulaciones no trabajan directamente con valores uniformes, sino con tiempos entre llegadas, cantidades de eventos, demandas, tiempos de servicio, fallas, éxitos o variables físicas que responden a otras distribuciones.

El problema central es entonces transformar una secuencia uniforme $U(0, 1)$ en observaciones de una distribución deseada. Naylor presenta este enfoque como la base para generar valores de variables aleatorias en simulación computacional: partir de números uniformes y aplicar funciones matemáticas o algoritmos específicos para obtener la distribución buscada [1]. En particular, pueden utilizarse técnicas como la transformación inversa, el método de rechazo, el método de composición o la simulación directa del experimento probabilístico que da origen a la variable.

El presente informe implementa y evalúa generadores para distribuciones continuas y discretas. En cada caso se presenta la función de probabilidad o densidad, sus parámetros, media y varianza, la función acumulada y el criterio utilizado para obtener o aproximar la generación de valores. El programa fue escrito en Python 3.x y genera automáticamente gráficas y una tabla resumen para contrastar el comportamiento simulado con el esperado.

2 Métodos

Todos los generadores parten de números pseudoaleatorios uniformes $U \sim U(0,1)$ provistos por `random.random()`. Se utilizó una semilla fija (42) para reproducir exactamente los resultados y $n = 10\,000$ muestras por distribución.

2.1 Transformación inversa y método de rechazo

La técnica de transformación inversa parte de la función de distribución acumulada $F(x)$ de una variable aleatoria X. Si $U \sim U(0,1)$, entonces se busca x tal que:

$$U = F(x) \tag{1}$$

Cuando F es invertible, la variable puede generarse como:

$$X = F^{-1}(U) \tag{2}$$

En las distribuciones discretas no existe una inversa continua, pero se puede aplicar el mismo principio acumulando probabilidades: se selecciona el primer valor x_i tal que $U \leq F(x_i)$.

Como alternativa general, el método de rechazo permite generar valores de una densidad $f(x)$ cuando resulta difícil invertir su acumulada. Si la variable está definida en un intervalo finito (a, b) y existe una constante c tal que $cf(x) \leq 1$, se generan pares uniformes y se acepta el candidato cuando cae debajo de la densidad escalada. Una forma usual de expresarlo es:

$$x = a + (b - a)U_1 \tag{3}$$

$$U_2 \leq cf(x) \tag{4}$$

Si la condición se cumple, x se acepta; si no, se rechaza y se genera un nuevo par de uniformes. En este trabajo se priorizaron métodos directos o propios de cada distribución por ser más simples y eficientes para los casos elegidos, pero el método de rechazo se considera como alternativa teórica general.

2.2 Distribución uniforme

La distribución uniforme continua en (a, b) posee densidad:

$$f(x) = \begin{cases} \frac{1}{b-a}, & a \leq x \leq b \\ 0, & \text{en otro caso} \end{cases} \tag{5}$$

Su acumulada es:

$$F(x) = \frac{x - a}{b - a} \tag{6}$$

Aplicando transformación inversa:

$$U = \frac{x - a}{b - a} \quad \Rightarrow \quad X = a + (b - a)U \tag{7}$$

Su media y varianza son:

$$E[X] = \frac{a + b}{2}, \qquad Var(X) = \frac{(b - a)^2}{12} \tag{8}$$

En la simulación se utilizó $a = 2$ y $b = 8$.

2.3 Distribución exponencial

La distribución exponencial modela tiempos entre eventos independientes con tasa λ:

$$f(x) = \lambda e^{-\lambda x}, \qquad x \geq 0 \tag{9}$$

Su función acumulada es:

$$F(x) = 1 - e^{-\lambda x} \tag{10}$$

La inversa se obtiene igualando $U = F(x)$:

$$U = 1 - e^{-\lambda x} \quad \Rightarrow \quad X = -\frac{\ln(1-U)}{\lambda} \tag{11}$$

Como $1 - U$ también es uniforme, suele escribirse $X = -\ln(U)/\lambda$. Sus momentos teóricos son:

$$E[X] = \frac{1}{\lambda}, \qquad Var(X) = \frac{1}{\lambda^2} \tag{12}$$

En este trabajo se usó $\lambda = 1{,}5$.

2.4 Distribución gamma

La distribución gamma, en la parametrización de tasa, tiene densidad:

$$f(x) = \frac{\lambda^k x^{k-1} e^{-\lambda x}}{(k-1)!}, \qquad x \geq 0 \tag{13}$$

con:

$$E[X] = \frac{k}{\lambda}, \qquad Var(X) = \frac{k}{\lambda^2} \tag{14}$$

Para k entero positivo se obtiene el caso Erlang. La función acumulada puede expresarse como:

$$F(x) = 1 - e^{-\lambda x} \sum_{i=0}^{k-1} \frac{(\lambda x)^i}{i!} \tag{15}$$

En general, esta acumulada no posee una inversa elemental simple. Por eso, para el caso particular de k entero, se utilizó la interpretación de la gamma/Erlang como suma de k variables exponenciales independientes:

$$X = \sum_{i=1}^{k} -\frac{\ln(1-U_i)}{\lambda} \tag{16}$$

En la simulación se empleó $k = 3$ y $\lambda = 1{,}2$.

2.5 Distribución normal

La distribución normal con media μ y desvío estándar σ tiene densidad:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \tag{17}$$

Su función acumulada se escribe como:

$$F(x) = \Phi\left(\frac{x-\mu}{\sigma}\right) \tag{18}$$

donde Φ es la acumulada de la normal estándar. Esta función no tiene inversa elemental cerrada, por lo que en este trabajo se utilizó el método directo de Box-Muller. Dados dos uniformes independientes U_1 y U_2:

$$Z_1 = \sqrt{-2\ln(U_1)}\cos(2\pi U_2), \qquad Z_2 = \sqrt{-2\ln(U_1)}\sin(2\pi U_2) \tag{19}$$

producen dos normales estándar. Luego se transforma:

$$X = \mu + \sigma Z \tag{20}$$

Sus momentos teóricos son:

$$E[X] = \mu, \qquad Var(X) = \sigma^2 \tag{21}$$

Se utilizaron $\mu = 10$ y $\sigma = 2$, es decir, una normal $N(10, 2^2)$.

2.6 Distribución de Pascal

La distribución de Pascal o binomial negativa modela la cantidad de fallas X antes de obtener r éxitos, con probabilidad de éxito p:

$$P(X = x) = \binom{r+x-1}{x} p^r (1-p)^x, \qquad x = 0, 1, 2, \ldots \tag{22}$$

Su función acumulada puede escribirse como suma de probabilidades puntuales:

$$F(x) = P(X \leq x) = \sum_{j=0}^{x} \binom{r+j-1}{j} p^r (1-p)^j \tag{23}$$

Por transformación inversa discreta, se podría generar el primer entero x tal que $U \leq F(x)$. En este trabajo se implementó una simulación equivalente del experimento: realizar ensayos de Bernoulli hasta acumular r éxitos y contar la cantidad de fallas observadas antes de alcanzarlos.

$$E[X] = \frac{r(1-p)}{p}, \qquad Var(X) = \frac{r(1-p)}{p^2} \tag{24}$$

En el experimento se usó $r = 4$ y $p = 0{,}35$.

2.7 Distribución binomial

La distribución binomial cuenta la cantidad de éxitos en m ensayos independientes de Bernoulli con probabilidad p:

$$P(X = x) = \binom{m}{x} p^x (1-p)^{m-x}, \qquad x = 0, 1, \ldots, m \tag{25}$$

Su acumulada es:

$$F(x) = P(X \leq x) = \sum_{j=0}^{x} \binom{m}{j} p^j (1-p)^{m-j} \tag{26}$$

También puede generarse por transformación inversa discreta seleccionando el primer x tal que $U \leq F(x)$. En este trabajo se eligió una implementación más directa: simular los m ensayos de Bernoulli y contar cuántos uniformes son menores que p.

$$E[X] = mp, \qquad Var(X) = mp(1-p) \tag{27}$$

Se generó realizando $m = 12$ ensayos por observación y contando los uniformes menores a $p = 0{,}4$.

2.8 Distribución hipergeométrica

La distribución hipergeométrica representa extracciones sin reemplazo desde una población de tamaño N, con K elementos exitosos y n extracciones realizadas:

$$P(X = x) = \frac{\binom{K}{x}\binom{N-K}{n-x}}{\binom{N}{n}} \tag{28}$$

Su acumulada se obtiene sumando la función de probabilidad hasta el valor x:

$$F(x) = P(X \leq x) = \sum_{j=0}^{x} \frac{\binom{K}{j}\binom{N-K}{n-j}}{\binom{N}{n}} \tag{29}$$

La transformación inversa discreta podría aplicarse sobre esta acumulada. En la implementación se simuló el experimento sin reemplazo: después de cada extracción se actualizan las cantidades restantes de éxitos y fracasos.

$$E[X] = n\frac{K}{N}, \qquad Var(X) = n\frac{K}{N}\left(1 - \frac{K}{N}\right)\frac{N-n}{N-1} \tag{30}$$

Se usó $N = 50$, $K = 18$ y $n = 10$.

2.9 Distribución de Poisson

La distribución de Poisson modela la cantidad de eventos en un intervalo fijo cuando estos ocurren con tasa promedio λ:

$$P(X = x) = \frac{e^{-\lambda}\lambda^x}{x!}, \qquad x = 0, 1, 2, \ldots \tag{31}$$

Su función acumulada es:

$$F(x) = P(X \leq x) = \sum_{j=0}^{x} \frac{e^{-\lambda}\lambda^j}{j!} \tag{32}$$

Por transformación inversa discreta podría seleccionarse el primer x tal que $U \leq F(x)$. En este trabajo se utilizó el método multiplicativo: se generan uniformes y se acumula su producto hasta que cae por debajo de $e^{-\lambda}$. El número de factores menos uno es el valor de Poisson.

$$E[X] = \lambda, \qquad Var(X) = \lambda \tag{33}$$

Se usó $\lambda = 4$.

2.10 Distribución empírica discreta

Una distribución empírica discreta asigna probabilidades explícitas a un conjunto finito de valores. En este trabajo:

x	1	2	3	4	5
$P(X = x)$	0,10	0,20	0,35	0,25	0,10

La función acumulada se construye sumando las probabilidades:

$$F(x_i) = \sum_{j=1}^{i} P(X = x_j) \tag{34}$$

El método de generación consiste en seleccionar el primer valor cuyo acumulado supere al uniforme generado:

$$X = x_i \quad \text{si} \quad F(x_{i-1}) < U \leq F(x_i) \tag{35}$$

Sus momentos se calcularon mediante:

$$E[X] = \sum_i x_i p_i, \qquad Var(X) = \sum_i x_i^2 p_i - (E[X])^2 \tag{36}$$

Para los valores elegidos, $E[X] = 3{,}05$ y $Var(X) = 1{,}2475$.

3 Implementación en Python

El Código 1 muestra las transformaciones principales para las distribuciones continuas. Todas parten de valores uniformes generados por `random.random()`.

Listing 1: Generadores continuos implementados en Python.

```python
def uniforme(n, a=2.0, b=8.0):
    return [a + (b - a) * random.random() for _ in range(n)]

def exponencial(n, lambd=1.5):
    return [-math.log(1 - random.random()) / lambd for _ in range(n)]

def gamma_erlang(n, k=3, lambd=1.2):
    valores = []
    for _ in range(n):
        total = 0.0
        for _ in range(k):
            total += -math.log(1 - random.random()) / lambd
        valores.append(total)
    return valores

def normal_box_muller(n, media=10.0, desvio=2.0):
    valores = []
    while len(valores) < n:
        u1 = max(random.random(), 1e-12)
        u2 = random.random()
        radio = math.sqrt(-2 * math.log(u1))
        z1 = radio * math.cos(2 * math.pi * u2)
        z2 = radio * math.sin(2 * math.pi * u2)
        valores.append(media + desvio * z1)
        if len(valores) < n:
            valores.append(media + desvio * z2)
    return valores
```

El Código 2 resume la lógica de generación para Pascal, binomial y Poisson. En Pascal y binomial se emplean ensayos de Bernoulli; en Poisson, el método multiplicativo.

Listing 2: Generadores discretos por ensayos y método multiplicativo.

```
def pascal(n, r=4, p=0.35):
    valores = []
    for _ in range(n):
        exitos = 0
        fallas = 0
        while exitos < r:
            if random.random() < p:
                exitos += 1
            else:
                fallas += 1
        valores.append(fallas)
    return valores

def binomial(n, ensayos=12, p=0.4):
    valores = []
    for _ in range(n):
        exitos = 0
        for _ in range(ensayos):
            if random.random() < p:
                exitos += 1
        valores.append(exitos)
    return valores

def poisson(n, lambd=4.0):
    valores = []
    limite = math.exp(-lambd)
    for _ in range(n):
        k = 0
        producto = 1.0
        while producto > limite:
            k += 1
            producto *= random.random()
        valores.append(k - 1)
    return valores
```

El Código 3 incluye los generadores de la distribución hipergeométrica y de la distribución empírica discreta, para completar el código de todas las distribuciones estudiadas.

Listing 3: Generadores discretos por muestreo sin reemplazo y acumulación.

```python
def hipergeometrica(n, poblacion=50, exitos_poblacion=18, extracciones=10):
    valores = []
    for _ in range(n):
        exitos_restantes = exitos_poblacion
        fracasos_restantes = poblacion - exitos_poblacion
        exitos = 0
        for _ in range(extracciones):
            total_restante = exitos_restantes + fracasos_restantes
            if random.random() < exitos_restantes / total_restante:
                exitos += 1
                exitos_restantes -= 1
            else:
                fracasos_restantes -= 1
        valores.append(exitos)
    return valores

def empirica_discreta(
    n,
    valores=(1, 2, 3, 4, 5),
    probabilidades=(0.10, 0.20, 0.35, 0.25, 0.10),
):
    acumuladas = []
    total = 0.0
    for prob in probabilidades:
        total += prob
        acumuladas.append(total)

    muestras = []
    for _ in range(n):
        u = random.random()
        for valor, acumulada in zip(valores, acumuladas):
            if u <= acumulada:
                muestras.append(valor)
                break
    return muestras
```

4 Criterios de validación

Para validar las distribuciones simuladas se compararon la media y la varianza muestral contra sus valores teóricos. Además, se aplicaron pruebas de bondad de ajuste según el tipo de distribución.

Para las distribuciones continuas se utilizó la prueba de Kolmogorov-Smirnov, comparando la función acumulada empírica $F_n(x)$ con la función acumulada teórica $F(x)$:

$$D = \sup_x |F_n(x) - F(x)| \tag{37}$$

Para $n = 10\,000$ y un nivel de significancia aproximado $\alpha = 0{,}05$, se tomó como criterio:

$$D < \frac{1{,}36}{\sqrt{n}} = 0{,}01360 \tag{38}$$

Para las distribuciones discretas se utilizó una prueba chi-cuadrado de bondad de ajuste, comparando frecuencias observadas O_i y esperadas E_i:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i} \tag{39}$$

Se consideraron las categorías con frecuencia esperada mayor o igual a 5, criterio usual para aplicar la aproximación chi-cuadrado. Dado que los parámetros de las distribuciones fueron fijados antes de simular y no estimados a partir de la muestra, los grados de libertad considerados fueron $gl = k - 1$, donde k es la cantidad de categorías incluidas en el contraste.

Table 1: Criterios de aceptación utilizados para las pruebas de bondad de ajuste.

Distribución	**Prueba**	**Criterio de aceptación**
Uniforme	KS	$D < 0{,}01360$
Exponencial	KS	$D < 0{,}01360$
Gamma	KS	$D < 0{,}01360$
Normal	KS	$D < 0{,}01360$
Pascal	χ^2, $gl = 27$	$\chi^2 < 40{,}11327$
Binomial	χ^2, $gl = 10$	$\chi^2 < 18{,}30704$
Hipergeométrica	χ^2, $gl = 8$	$\chi^2 < 15{,}50731$
Poisson	χ^2, $gl = 12$	$\chi^2 < 21{,}02607$
Empírica discreta	χ^2, $gl = 4$	$\chi^2 < 9{,}48773$

5 Resultados y discusión

La Tabla 2 presenta la comparación entre valores observados y teóricos para $10\,000$ muestras. En distribuciones continuas, el último estadístico corresponde a Kolmogorov-Smirnov; en distribuciones discretas, corresponde a chi-cuadrado. La columna "Resultado" indica si el estadístico quedó dentro del criterio de aceptación definido en la Tabla 1.

Table 2: Resumen de resultados de la simulación con semilla 42.

Distribución	**Media obs.**	**Media teo.**	**Var. obs.**	**Var. teo.**	**KS/χ^2**	**Resultado**
Uniforme	5,00151	5,00000	2,97963	3,00000	0,00626	OK
Exponencial	0,67046	0,66667	0,43472	0,44444	0,00984	OK
Gamma	2,48818	2,50000	2,06080	2,08333	0,01039	OK
Normal	10,02792	10,00000	3,99574	4,00000	0,01046	OK
Pascal	7,42330	7,42857	20,91281	21,22449	31,18428	OK
Binomial	4,77190	4,80000	2,83935	2,88000	8,76653	OK
Hipergeométrica	3,60420	3,60000	1,90193	1,88082	4,72206	OK
Poisson	4,00470	4,00000	3,97608	4,00000	13,13614	OK
Empírica discreta	3,05630	3,05000	1,24505	1,24750	2,57414	OK

Los errores relativos fueron bajos en todos los casos. La media observada de la uniforme fue $5{,}00151$, prácticamente igual al valor teórico 5. La exponencial y la gamma muestran pequeñas diferencias atribuibles al error muestral. La normal generada por Box-Muller también coincide de forma clara con la media y varianza esperadas.

En las distribuciones discretas, la comparación mediante barras simuladas y probabilidades teóricas permite verificar visualmente el ajuste. La binomial, hipergeométrica, Poisson y empírica discreta se comportan de manera muy estable. La distribución de Pascal presenta mayor dispersión, algo esperable por su varianza elevada, pero aun así su estadístico chi-cuadrado queda dentro del criterio de aceptación definido para los grados de libertad considerados.

5.1 Gráficas de distribuciones continuas

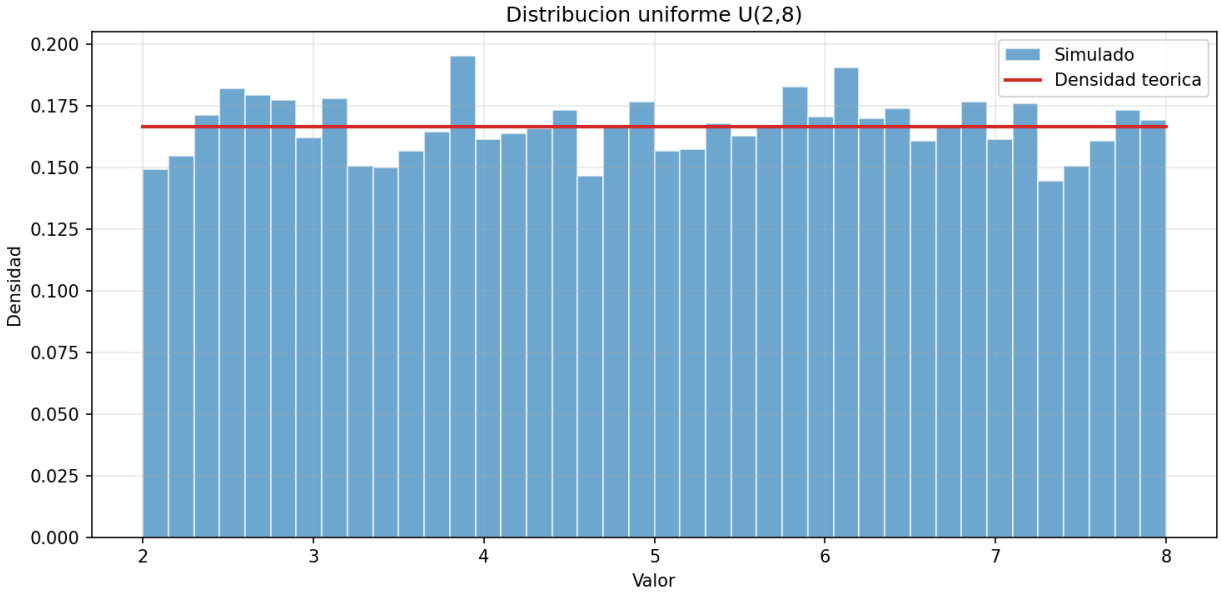


Figure 1: Valores generados para una distribución uniforme $U(2, 8)$.

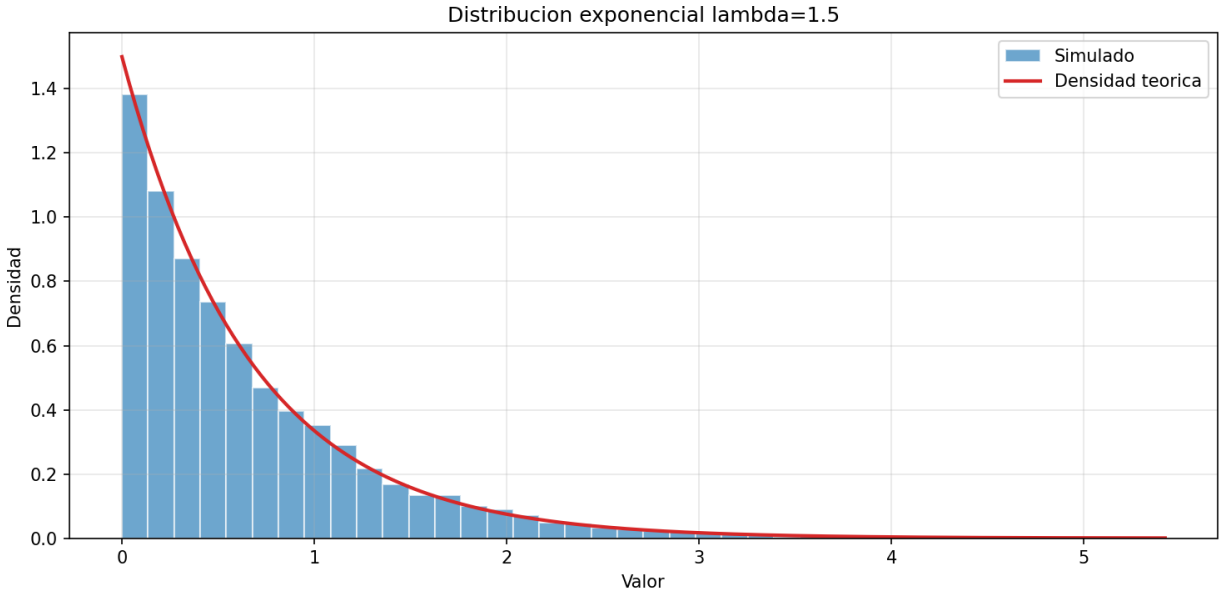


Figure 2: Valores generados para una distribución exponencial con $\lambda = 1{,}5$.

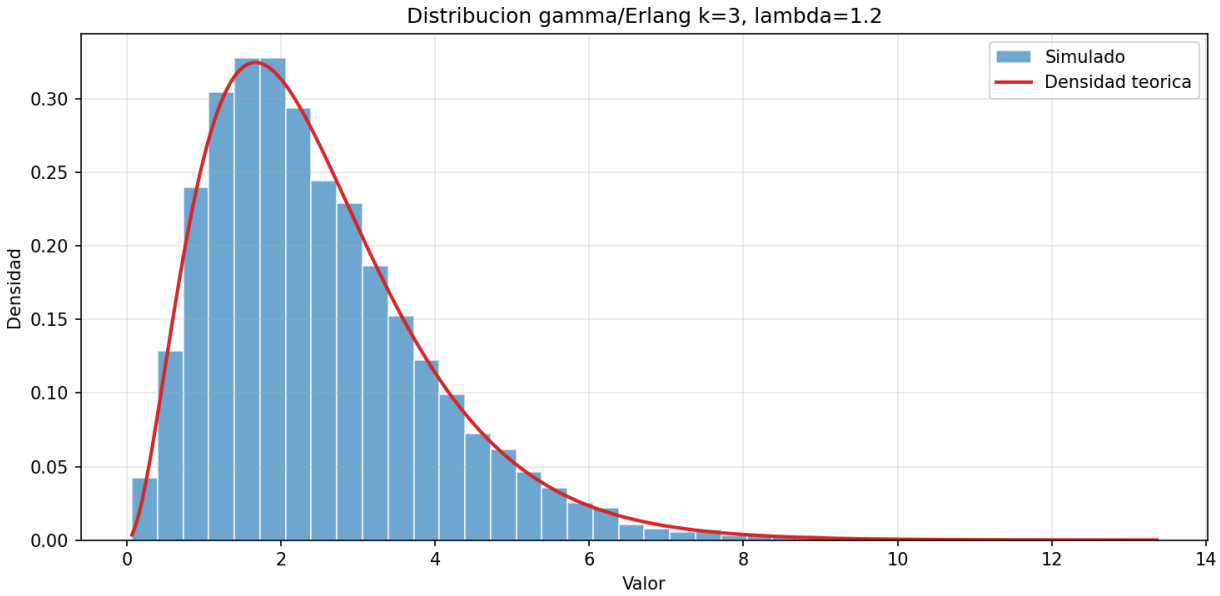


Figure 3: Valores generados para una distribución gamma/Erlang con $k = 3$ y $\lambda = 1{,}20$.

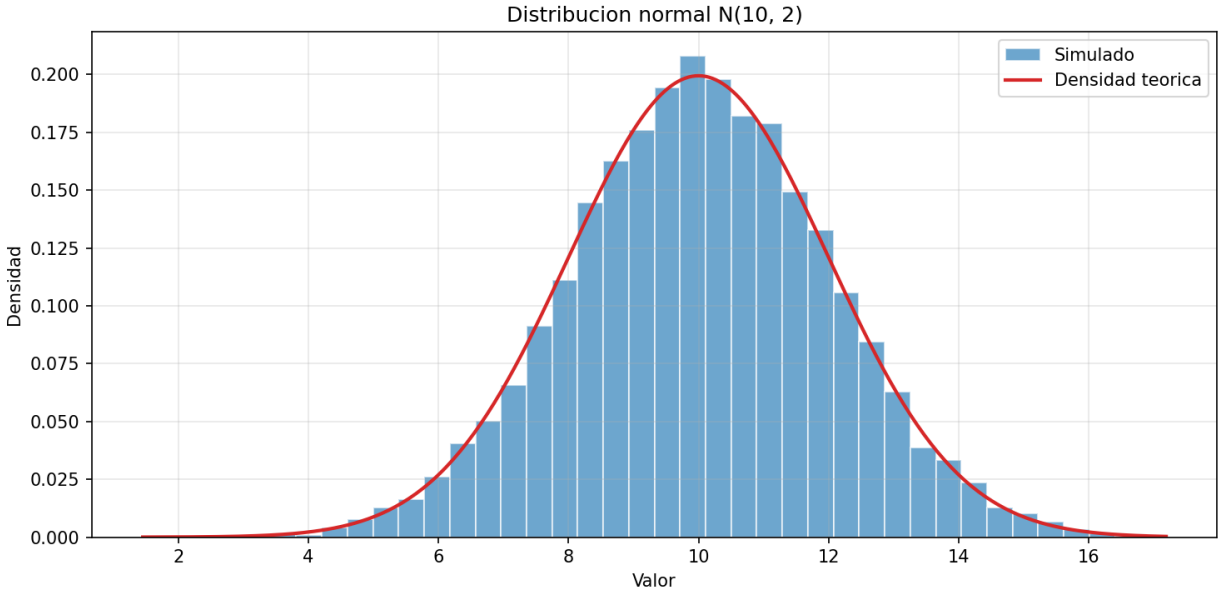


Figure 4: Valores generados para una distribución normal $N(10, 2^2)$.

5.2 Gráficas de distribuciones discretas

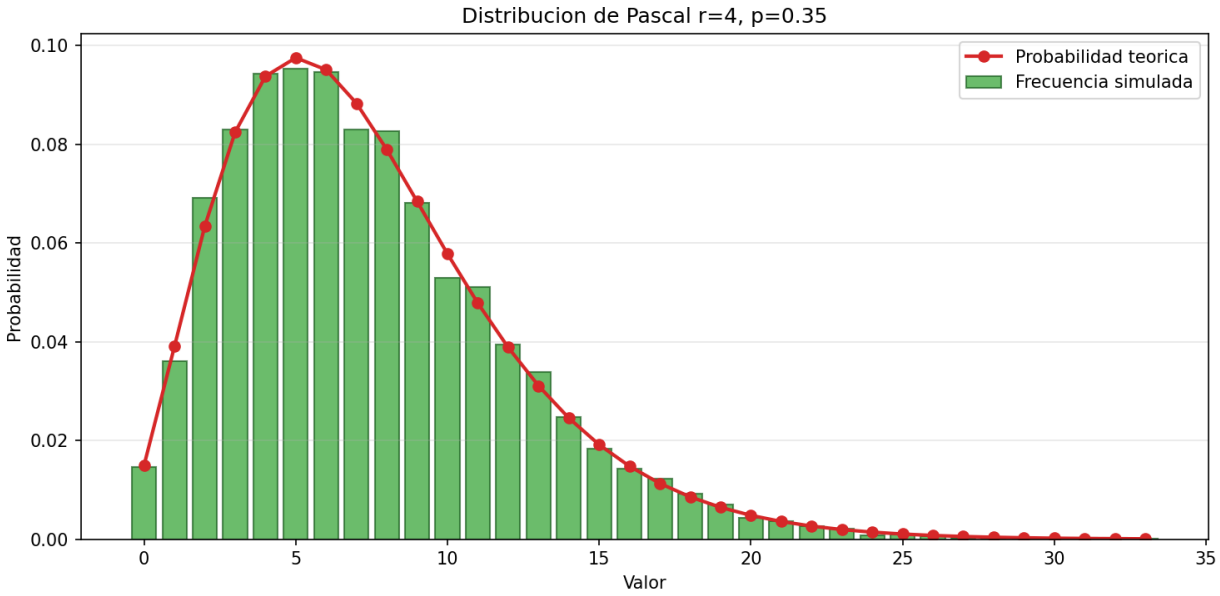


Figure 5: Valores generados para una distribución de Pascal con $r = 4$ y $p = 0{,}35$.

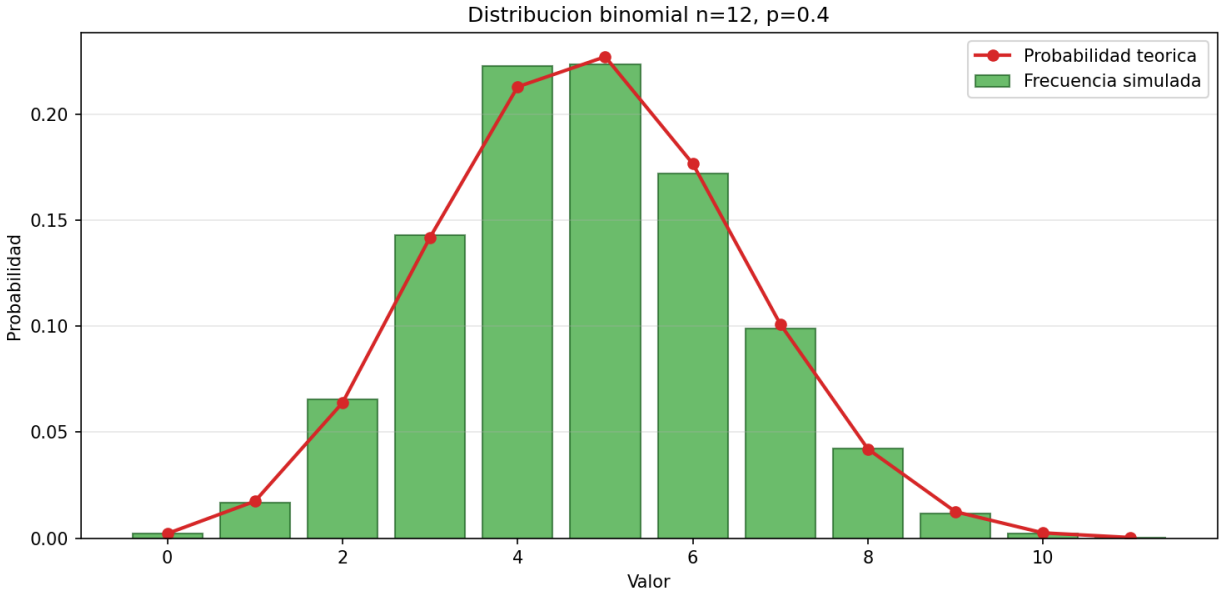


Figure 6: Valores generados para una distribución binomial con $m = 12$ y $p = 0{,}40$.

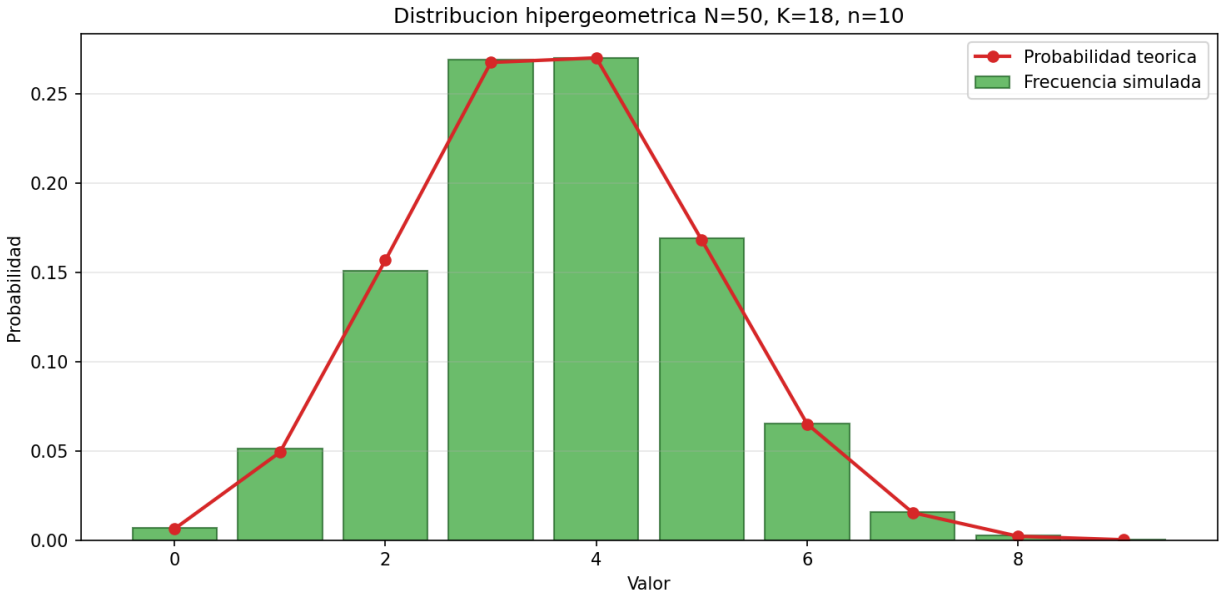


Figure 7: Valores generados para una distribución hipergeométrica con $N = 50$, $K = 18$ y $n = 10$.

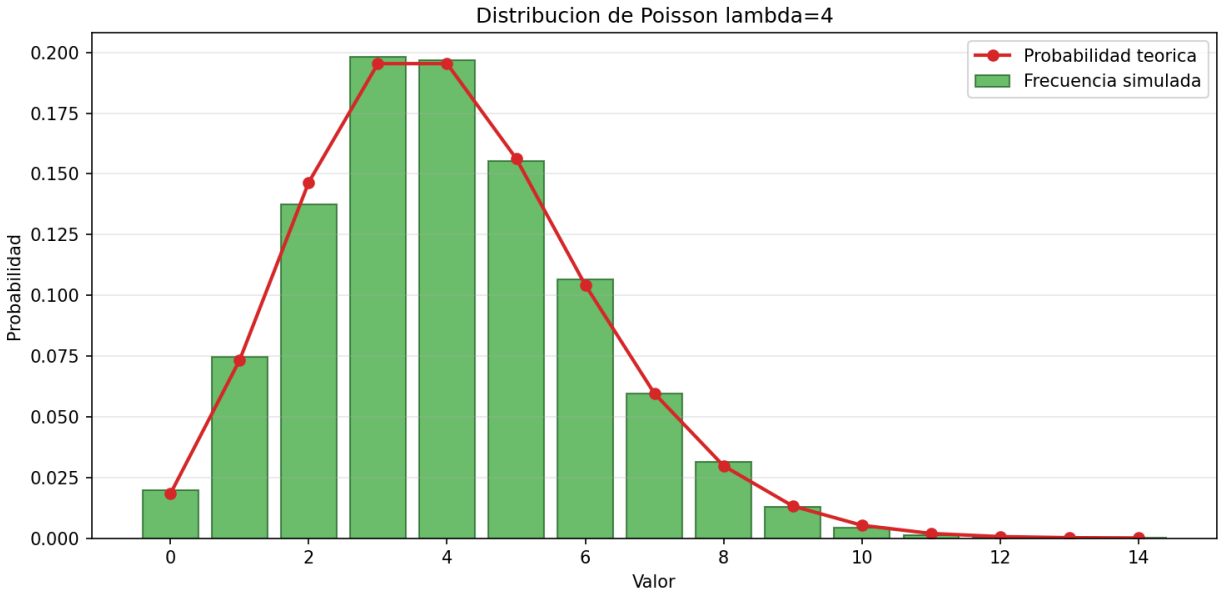


Figure 8: Valores generados para una distribución de Poisson con $\lambda = 4$.

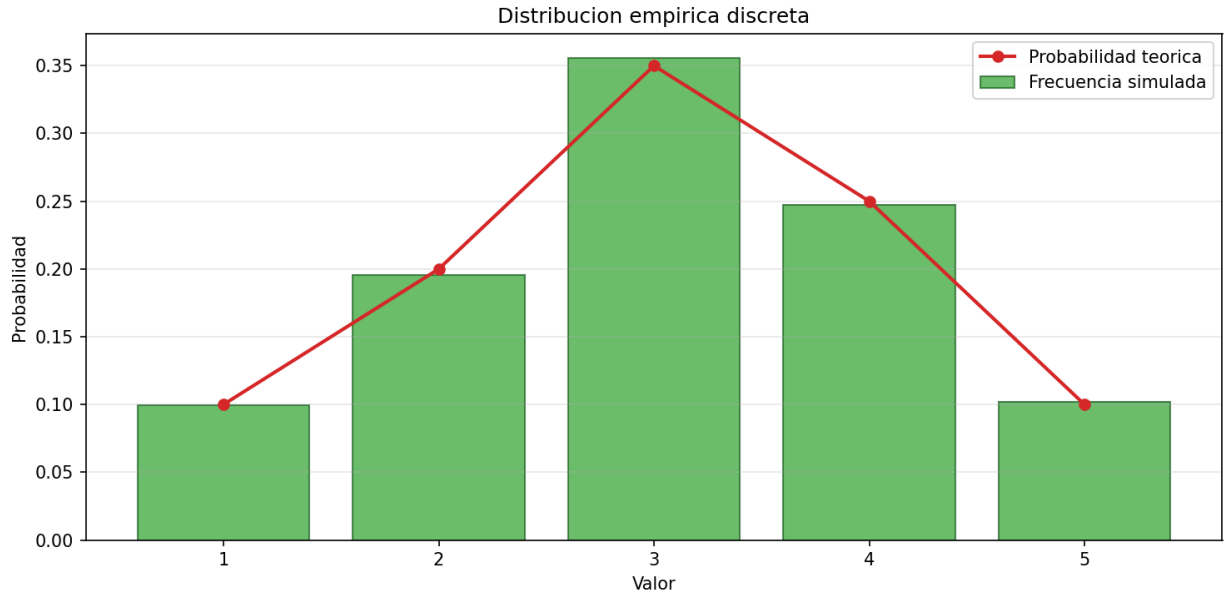


Figure 9: Valores generados para una distribución empírica discreta.

5.3 Comparación global

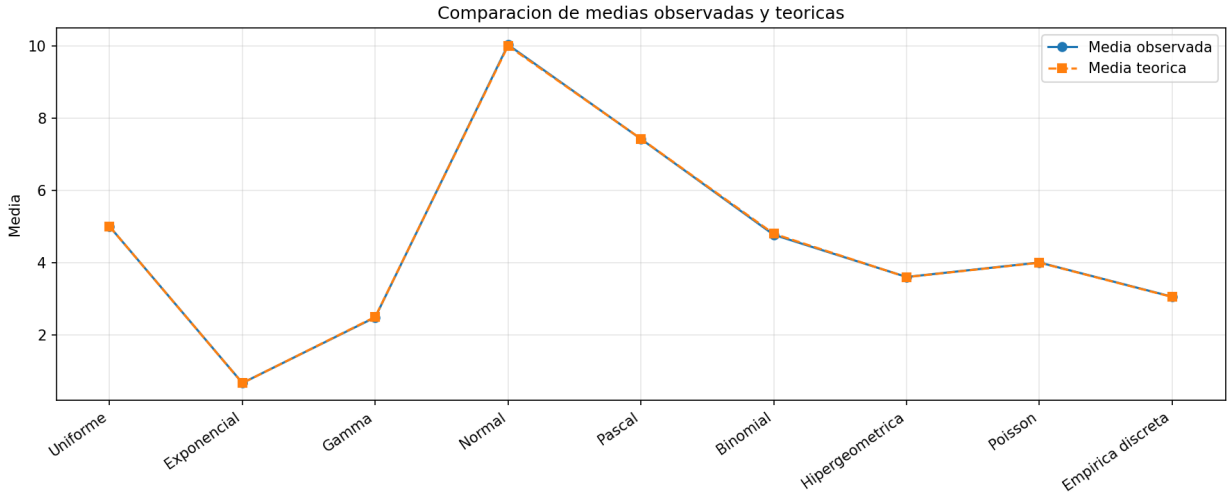


Figure 10: Comparación entre medias observadas y medias teóricas.

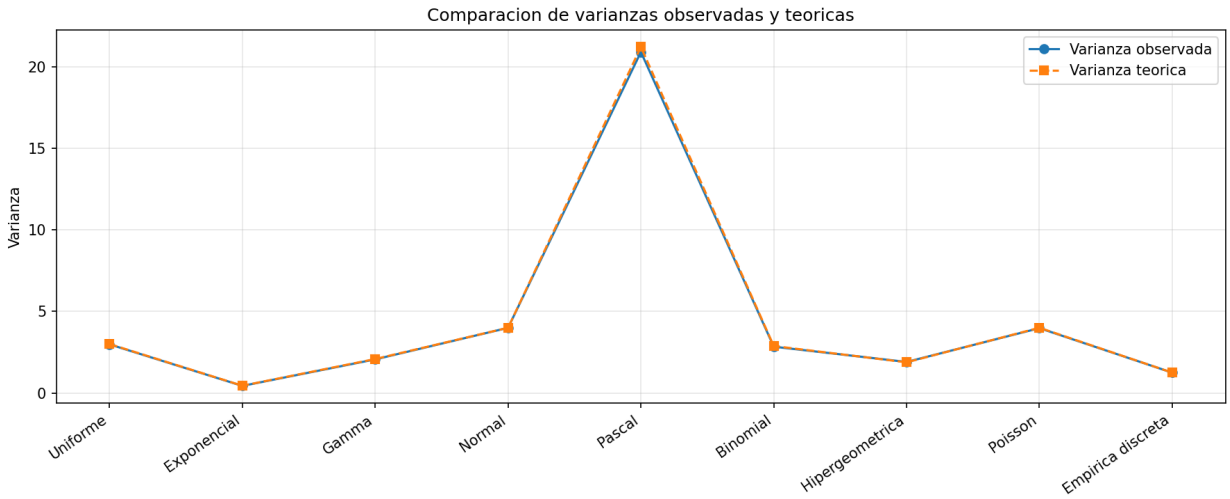


Figure 11: Comparación entre varianzas observadas y varianzas teóricas.

6 Conclusiones

El trabajo demuestra que una fuente uniforme validada puede transformarse en variables aleatorias de múltiples distribuciones. La transformación inversa resultó directa para la uniforme y la exponencial; la gamma se generó como suma de exponenciales; la normal mediante Box-Muller; y las distribuciones discretas mediante ensayos, acumulación de probabilidades o mecanismos propios del experimento probabilístico.

Los resultados obtenidos fueron consistentes con la teoría. Las medias y varianzas simuladas se mantuvieron cercanas a sus valores esperados, y las gráficas muestran una buena coincidencia entre las frecuencias observadas y las funciones teóricas. Además, los estadísticos de Kolmogorov-Smirnov para las distribuciones continuas y los estadísticos chi-cuadrado para las distribuciones discretas quedaron dentro de los criterios de aceptación definidos.

También se observa que cada distribución requiere una técnica de generación distinta. No existe un único procedimiento universal igualmente simple para todos los casos: algunas distribuciones surgen por inversión de la acumulada, otras por suma o composición de variables, otras por aceptación y rechazo, y otras por simular el proceso físico o lógico que les da origen.

En síntesis, el TP confirma que la generación de variables aleatorias para simulación no consiste solamente en producir números uniformes, sino en aplicar transformaciones matemáticas coherentes con la distribución que se desea representar y validar luego que los datos generados respeten razonablemente sus propiedades teóricas.

References

[1] T. H. Naylor, *Técnicas de simulación en computadoras*, Editorial Limusa, 1982.

[2] J. I. Torres, *TP 2.2 - Generadores de números pseudoaleatorios de distintas distribuciones de probabilidad*, Universidad Tecnológica Nacional - FRRO, 2026.

[3] Python Software Foundation, *random — Generate pseudo-random numbers*, `https://docs.python.org/3/library/random.html`.

[4] G. E. P. Box and M. E. Muller, "A Note on the Generation of Random Normal Deviates", *The Annals of Mathematical Statistics*, vol. 29, no. 2, pp. 610–611, 1958.

[5] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, Addison-Wesley, 1997.